



Реагирование на компьютерные инциденты



СОПЕВУ

Урок 15

Форензика в Linux

Оглавление

Описание устройства файловой системы ext4 и работы с ней посредством TSK	3
Анализ журналов событий Linux.....	15
Описание механизмов закрепления Linux.....	20
Резюме урока	25



Описание устройства файловой системы ext4 и работы с ней посредством TSK

Файловая система EXT (Extended File System) была разработана специально для операционной системы Linux. Главной целью, которую преследовали создатели данной ФС, было преодоление **максимального размера записываемых файлов**, который в то время составлял всего 64 МБ.

Благодаря созданию новой **структуры метаданных** – максимально возможный размер файла увеличился до **2 ГБ**. В то же время **максимальная длина** имен файлов увеличилась до **255 байт**.

Хоть EXT и преодолела основные недостатки файловой системы Minix (которая использовалась до этого в Linux) ее главным недостатком были **временные метки**. В EXT разрешалось использовать **только одну временную метку** для каждого файла.

Спустя ряд изменений разных версий EXT появилась версия **EXT4**, которая получила **ряд преимуществ** по сравнению с предыдущими версиями, среди которых:

пространственная запись файлов, которая используется для увеличения быстродействия файловой системы. Перед тем, как записать файл на



диск – система выделяет нужную область на диске и после этого данные записываются в конец этой области;

обратная совместимость с Ext2 и Ext3 (например, раздел Ext3 может быть смонтирован с помощью драйвера Ext4);

использование экстентов – в старых версиях отображение блоков данных файла было реализовано старым способом – то есть, отображаются все блоки, относящиеся к конкретному файлу. Это накладывает некоторые ограничения при работе с файлами большого размера. Внедрение экстентов позволило выводить большое количество последовательных блоков информации при помощи одного дескриптора. Такой подход увеличивает производительность файловой системы в несколько раз, так как система сохраняет только адрес первого и последнего блока данных, которые соответствуют большому файлу;

уменьшение фрагментации файлов за счет более рационального выделения блоков памяти - перед тем, как записать файл Ext4 выделяет блоки, которые находятся поблизости, чтобы сократить время поиска нужного блока во время чтения данных

отложенное выделение – блоки памяти выделяются непосредственно перед записью файлов на диск. Такой подход позволяет снизить нагрузку на кэш-память и соответственно увеличить производительность;

использование контрольных сумм журналов ФС — Ext4 постоянно проверяет блоки данных на наличие повреждений. В свою очередь это



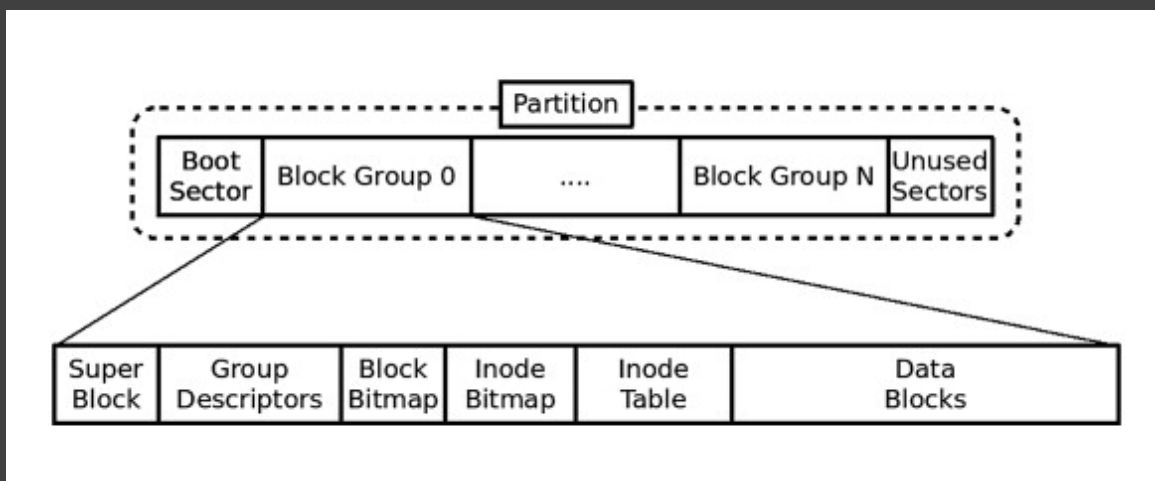
сокращает время журналирования, что увеличивает производительность.

Главными сущностями файловой системы Ext4 являются **блоки данных** (data blocks) и **индексные дескрипторы** (inodes).

Блоки данных представляют собой участки дискового пространства, в которых могут храниться **данные содержимого файлов** файловой системы.

Inodes предназначены для **описания метаданных файла и используемых для него блоков**. Inodes не имеют имен, а только идентификатор. Сопоставление **имени файла** и его **inode** находится в структуре каталога.

Схематически структура файловой системы **Ext4** может быть представлена следующим образом:



Структура файловой системы Ext4



Раздел (Partition) Ext4 может содержать загрузочный сектор (Boot Sector), группы блоков (Block Group) и незадействованные сектора (Unused Blocks).

При этом компоненты группы блоков могут быть описаны следующим образом:

Super Block – находится в самом начале файловой системы (обычно в первых 1024 байтах раздела). Система автоматически создает несколько копий суперблока, так как без него она не сможет функционировать. В суперблоке хранится базовая информация о файловой системе, а именно:

- общее число блоков данных и inodes для всей файловой системы;
- количество свободных inodes и блоков данных, в которые можно будет записать файлы;
- размер inode и блока данных (эти данные указываются при создании файловой системы);
- информация о файловой системе – время монтирования, последние изменения и т.д.

Group Description Table (глобальная таблица дескрипторов группы блоков) расположена сразу же после суперблока. В ней описаны первый и последний блоки для каждой группы блоков, а также информация где именно в каждой группе начинается таблица inodes, начало блоков данных и т.д.



Block Bitmap (битовая карта блока) – это специальная таблица, в которой указано, какие блоки в группе **использованы**, а какие **свободны**. Эта информация используется во время **распределения информации** в блоке. При этом в карте 0 – означает что блок свободен, а 1 – что занят.

Inode Bitmap (битовая карта inodes) – эта таблица аналогична **битовой карте блока**, только в ней отображается информация о свободных **inodes**, которые могут быть использованы для записи **новых файлов**.

Inode Table (таблица inodes) – используется для описания используемых **inodes**, которые, в свою очередь, описывают **метаданные файла** и **расположение блоков данных** файла.

Блоки данных – выделенные физические блоки памяти, в которых хранятся данные самих файлов.

Для **криминалистического** анализа файловой системы Ext4 может применяться набор инструментов **The Sleuth Kit**. Данный набор также поддерживает **перечень других популярных ФС** (TFS, FAT, ExFAT, UFS 1, UFS 2, EXT2FS, EXT3FS, HFS, ISO 9660 и YAFFS2).

Оригинальная часть **The Sleuth Kit** – это **библиотека С** и набор инструментов с **интерфейсом командной строки** для криминалистического анализа файлов, дисков и томов.



Инструменты TSK могут быть выделены в следующие группы:

Автоматизированные инструменты файловой системы;

Инструмент уровня файловой системы (fsstat);

Инструменты уровня имени файла;

Инструменты уровня метаданных;

Инструменты уровня единиц данных;

Инструменты журнала файловой системы;

Инструменты системных разделов;

Инструменты для работы с файлами образов;

Другие инструменты.

Автоматизированные инструменты файловой системы объединяют функциональность тома и файловой системы. Вместо того чтобы анализировать только одну файловую систему, эти инструменты принимают образ диска в качестве входных данных, идентифицируют тома и обрабатывают содержимое.

Среди данных инструментов:

`tsk_comparedir` - сравнивает иерархию локальных каталогов с содержимым необработанного устройства (или образа диска). Это можно использовать для обнаружения руткитов, которые скрывают присутствие вредоносных модулей.

`tsk_gettimes` - извлекает все данные связанные со временем из образа для создания временной шкалы.

`tsk_loaddb` - загружает метаданные из образов в базу данных SQLite.

`tsk_recover` - извлекает невыделенные (или выделенные) файлы из образа диска в локальный каталог (в случае, если требуется провести углубленный анализ таких файлов).



Инструмент уровня файловой системы (`fsstat`) показывает детали и статистику файловой системы, включая разметку, размеры и метки. Использование `fsstat` позволит получить первичное представление о состоянии ФС, с которой ведется работа.

Пример описания файловой системы посредством `fsstat` представлен ниже:

```
root@ir:~# fsstat /dev/nvme0n1p2 | more
FILE SYSTEM INFORMATION
-----
File System Type: Ext4
Volume Name:
Volume ID: 29fa4b671dad17859d41566bcb2d6867

Last Written at: 2023-03-09 08:56:18 (MSK)
Last Checked at: 2021-06-09 10:19:33 (MSK)

Last Mounted at: 2023-03-09 08:56:19 (MSK)
Unmounted properly
Last mounted on: /

Source OS: Linux
Dynamic Structure
Compat Features: Journal, Ext Attributes, Resize Inode, Dir Index
InCompat Features: Filetype, Needs Recovery, Extents, 64bit, Flexible Block Groups,
Read Only Compat Features: Sparse Super, Large File, Huge File, Extra Inode Size

Journal ID: 00
Journal Inode: 8

METADATA INFORMATION
-----
Inode Range: 1 - 31244289
Root Directory: 2
Free Inodes: 30304426
Inode Size: 256
Orphan Inodes: 22548212, 22548207, 22548203, 22547811, 20724610, 22548208, 20731454, 2072
2269, 22547879, 22548205, 22547882, 22548204, 22547996, 22547816, 20722253, 22547881, 207
31459, 22548202, 20731041, 20731006, 22547807, 22547805, 20712256, 22548069, 20711877, 10
59860, 22547995, 22547809, 22545439, 22545304, 22547880, 22545300, 22547808, 22547806, 22
547803, 22547804, 22545349, 22545438, 20726116, 22547737, 2490569, 1069382, 1069376, 1069
361, 1069354, 1069349, 1069345, 1069341, 1069275, 1069274, 1069273, 1069271, 1069270, 106
9268, 1069266, 1069265, 1066883, 1066879, 1066875, 1065524, 1065501, 1048901, 22545436, 2
2545335, 22545358, 22545348, 22545353, 22545325, 20725022, 22544820, 22545334, 22545341,
20720467, 20721501, 20720104, 22545306, 22544523, 22544399, 22545305, 22544522, 22545299,
20720234, 22544004, 22544546, 22545304, 20720536, 22544403, 22544523, 22544524, 1105000
```



В выводе инструмента представлено общее описание раздела Ext4, включая время **последней записи**, **монтирования**, **характеристики inodes** и т.д. После чего следует описание групп блоков:

```
CONTENT INFORMATION
-----
Block Groups Per Flex Group: 16
Block Range: 0 - 124948991
Block Size: 4096
Free Blocks: 10915000

BLOCK GROUP INFORMATION
-----
Number of Block Groups: 3814
Inodes per group: 8192
Blocks per group: 32768

Group: 0:
  Block Group Flags: [INODE_ZEROED]
  Inode Range: 1 - 8192
  Block Range: 0 - 32767
  Layout:
    Super Block: 0 - 0
    Group Descriptor Table: 1 - 60
    Group Descriptor Growth Blocks: 61 - 1072
    Data bitmap: 1073 - 1073
    Inode bitmap: 1089 - 1089
    Inode Table: 1105 - 1616
    Data Blocks: 9297 - 32767
  Free Inodes: 8169 (99%)
  Free Blocks: 5405 (16%)
  Total Directories: 2
  Stored Checksum: 0x7835

Group: 1:
  Block Group Flags: [INODE_UNINIT, INODE_ZEROED]
  Inode Range: 8193 - 16384
  Block Range: 32768 - 65535
  Layout:
    Super Block: 32768 - 32768
    Group Descriptor Table: 32769 - 32828
    Group Descriptor Growth Blocks: 32829 - 33840
--More--
```



Инструменты уровня имени файла обрабатывают структуры имён файлов, которые обычно находятся в родительском каталоге.

Среди таких инструментов:

ffind - находит имена выделенных и нераспределенных файлов, которые указывают на заданную структуру метаданных.

fls - перечисляет имена выделенных и удалённых файлов в каталоге.

Использование данных инструментов позволяет получить представление об активности в отношении файлов на вовлеченном в инцидент устройстве и осуществить поиск интересующих файлов.

Инструменты уровня метаданных обрабатывают структуры метаданных, в которых хранятся сведения о файле. Примеры этой структуры включают записи каталога в FAT, записи MFT в NTFS и inodes в ExtX и UFS. Среди данных инструментов:

icat - извлекает блоки данных из файла, который определяется его адресом метаданных (вместо имени файла).

ifind - находит структуру метаданных, которая имеет указанное имя файла, указывающее на неё, или структуру метаданных, которая указывает на заданную единицу данных.

ils - перечисляет структуры метаданных и их содержимое в формате с разделителями каналов.

istat - отображает статистику и подробную информацию о данной структуре метаданных в удобном для чтения формате.



Инструменты уровня единиц данных обрабатывают блоки данных, в которых хранится содержимое файла (на примере с Ext4 это блоки данных). Среди таких инструментов:

blkcat - извлекает содержимое заданного блока данных.

blkls - перечисляет подробную информацию о единицах данных и может извлекать нераспределённое пространство файловой системы.

blkstat - отображает статистику о данном блоке данных в удобном для чтения формате

blkcalc - вычисляет, где данные в нераспределённом пространстве образа (из blkls) существуют в исходном образе.

Данная группа инструментов предназначена для **низкоуровневого анализа дисков**, вовлеченных в инцидент. Например, чтобы попытаться восстановить фрагменты удаленного вредоносного модуля.

Инструменты журнала файловой системы анализируют журналы журналируемых файловых систем (в том числе NTFS и Ext4). Среди таких инструментов:

jcat - отображение содержимого определенного блока журнала.

jls - список записей в журнале файловой системы.

Данные инструменты могут быть полезны при восстановлении файлов, а также при построении картины последней активности на хосте.

Инструменты системных разделов анализируют структуру разделов диска или образа диска. Среди данных инструментов:

mmls - отображает структуру диска, включая незанятое пространство.

mmstat - отображение сведений о системе томов (обычно только тип).



mmcat - извлекает содержимое определённого тома в STDOUT.

Пример вывода mmls:

```
root@ir:~# mmls /dev/nvme0n1
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors
```

	Slot	Start	End	Length	Description
000:	Meta	0000000000	0000000000	0000000001	Safety Table
001:	-----	0000000000	0000004095	0000004096	Unallocated
002:	Meta	0000000001	0000000001	0000000001	GPT Header
003:	Meta	0000000002	0000000033	0000000032	Partition Table
004:	000	0000004096	0000618495	0000614400	
005:	001	0000618496	1000210431	0999591936	
006:	-----	1000210432	1000215215	0000004784	Unallocated

Вывод mmls

Инструменты для работы с файлами образов включают следующие средства:

img_stat - инструмент покажет подробную информацию о формате образа

img_cat - этот инструмент покажет необработанное содержимое файла образа

Группа **других инструментов** включает следующие:

hfind - использует алгоритм двоичной сортировки для поиска хешей в NIST NSRL, Hashkeeper и пользовательских базах данных хешей, созданных md5sum.

mactime - принимает входные данные от инструментов fls и ils для создания шкалы времени активности файлов.

sorter - сортирует файлы по их типу и выполняет проверку расширений и поиск в базе данных хешей.



sigfind - ищет двоичное значение по заданному смещению. Полезно для восстановления потерянных структур данных.

Для лучшего освоения возможностей TSK при проведении РКИ рекомендуется ознакомиться с документацией:

<https://www.sleuthkit.org/sleuthkit/docs.php>

Резюме

Понимание основных принципов работы файловой системы Ext4 является полезным навыком при проведении криминалистического анализа вовлеченного в компьютерный инцидент устройства под управлением ОС Linux, так как наверняка на данном устройстве будет использоваться именно эта файловая система.

При этом для углубленного изучения диска такого устройства может применяться набор инструментов TSK, который помогает извлекать и анализировать информацию о содержимом файловой системы на разных уровнях абстракции.



Анализ журналов событий Linux

Анализ журналов событий Linux является таким же полезным и важным мероприятием, как и анализ журналов событий Windows или других событий, собираемых посредством SIEM.

Журналы событий Linux как правило хранятся в директории `/var/log`.

Все журналы в ОС Linux могут быть выделены в следующие группы:

журналы приложений;

журналы системных событий;

журналы служб.

Наиболее важными при проведении анализа событий являются следующие журналы:

`/var/log/auth.log`

`/var/log/wtmp`

`/var/log/syslog`

`/var/log/auth.log` - этот журнал содержит все события аутентификации и события сеанса задания Cron (например, запуск задачи, закрытие задачи и т. д.). Это может быть самым важным журналом для анализа, так как помогает понять, кто и откуда подключался к системе в период вредоносной активности.

В директории `/var/log`, помимо самого файла `auth.log`, хранятся также архивы нескольких предыдущих журналов `auth`:




```

root@ir:~# ls /var/log | grep auth
auth.log
auth.log.1
auth.log.2.gz
auth.log.3.gz
auth.log.4.gz
root@ir:~# █

```

Пример именования архивов auth.log

Примеры попыток неудачной аутентификации представлены на скриншоте:

```

root@adc1:~# egrep "Failed|failure" /var/log/auth.log
Dec  5 21:39:17 adc1 sshd[41458]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.3 user=root
Dec  5 21:39:20 adc1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec  5 21:39:23 adc1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec  5 21:39:28 adc1 sshd[41458]: Failed password for root from 192.168.1.3 port 37362 ssh2
Dec  5 21:39:28 adc1 sshd[41458]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh
ruser= rhost=192.168.1.3 user=root
Dec  5 21:39:41 adc1 sshd[41469]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.3 user=tecmint
Dec  5 21:39:44 adc1 sshd[41469]: Failed password for tecmint from 192.168.1.3 port 37364 ssh2
Dec  5 21:40:18 adc1 sshd[41491]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.245 user=root
Dec  5 21:40:21 adc1 sshd[41491]: Failed password for root from 192.168.1.245 port 52882 ssh2
Dec  5 21:40:24 adc1 sshd[41491]: Failed password for root from 192.168.1.245 port 52882 ssh2
Dec  5 21:40:30 adc1 sshd[41491]: Failed password for root from 192.168.1.245 port 52882 ssh2
Dec  5 21:40:30 adc1 sshd[41491]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh
ruser= rhost=192.168.1.245 user=root
Dec  5 21:40:42 adc1 sshd[41506]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
tty=ssh ruser= rhost=192.168.1.245 user=admin
Dec  5 21:40:42 adc1 sshd[41506]: pam_winbind(sshd:auth): request wbcLogonUser failed: WBC_ERR_AUTH_
ERROR, PAM error: PAM_AUTH_ERR (7), NTSTATUS: NT_STATUS_LOGON_FAILURE, Error message was: Logon fail
ure
Dec  5 21:40:45 adc1 sshd[41506]: Failed password for admin from 192.168.1.245 port 52884 ssh2
root@adc1:~# _

```

События auth.log



`/var/log/wtmp` - содержит записи о **включениях** и **перезагрузках** системы. Представлено в **бинарном формате**, при этом сами записи содержатся в **ascii**:

```

~ shutdown 5.3.0-28-generic
~ reboot 5.3.0-59-generic
~ reboot 5.3.0-59-generic
~ shutdown 5.4.0-37-generic
~ reboot 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ runlevel 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ runlevel 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ runlevel 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ reboot 5.4.0-37-generic
~ shutdown 5.4.0-37-generic
~ reboot 5.4.0-39-generic
~ shutdown 5.4.0-39-generic

```

Пример содержимого wtmp

`/var/log/syslog` - содержит основные **системные события** (в том числе может содержать информацию о выполнении заданий Cron). Как и **auth.log**, имеет архивы с более старыми событиями.

Пример содержимого представлен на скриншоте:



```

5183 Mar 12 00:17:01 ir kernel: [228042.663871] alx 0000:03:00.0: device [1969:e0b1] error status/mask=00000080/00002000
5184 Mar 12 00:17:01 ir kernel: [228042.663872] alx 0000:03:00.0: [ 7] BadDLLP
5185 Mar 12 00:17:01 ir CRON[223212]: (root) CMD ( cd / && run-parts --report /etc/cron.hourly)
5186 Mar 12 00:17:03 ir kernel: [228045.014632] pcieport 0000:00:1d.6: AER: Corrected error received: 0000:03:00.0
5187 Mar 12 00:17:03 ir kernel: [228045.014729] alx 0000:03:00.0: PCIe Bus Error: severity=Corrected, type=Data Link Layer, (Receiver ID)
5188 Mar 12 00:17:03 ir kernel: [228045.014737] alx 0000:03:00.0: device [1969:e0b1] error status/mask=00000080/00002000
5189 Mar 12 00:17:03 ir kernel: [228045.014746] alx 0000:03:00.0: [ 7] BadDLLP
5190 Mar 12 00:17:04 ir kernel: [228046.571751] pcieport 0000:00:1d.6: AER: Corrected error received: 0000:03:00.0
5191 Mar 12 00:17:04 ir kernel: [228046.571868] alx 0000:03:00.0: PCIe Bus Error: severity=Corrected, type=Physical Layer, (Receiver ID)
5192 Mar 12 00:17:04 ir kernel: [228046.571876] alx 0000:03:00.0: device [1969:e0b1] error status/mask=00000001/00002000
5193 Mar 12 00:17:04 ir kernel: [228046.571885] alx 0000:03:00.0: [ 0] RxErr

```

Содержимое syslog

События **syslog** могут собираться **различными системами мониторинга**, при этом конфигурация **syslog** определяется в следующих файлах:

`/etc/syslog.conf`

`/etc/rsyslog.conf`

`/etc/rsyslog.d/*.conf`

`/etc/syslog-ng.conf`

`/etc/syslog-ng/*`

Подробнее об анализе журналов событий Linux при проведении РКИ описано в книге Practical Linux Forensics (<https://nostarch.com/practical-linux-forensics>).

Резюме

Журналы событий Linux **не так содержательны** с точки зрения анализа вредоносной активности, как журналы Windows. Наиболее значимыми являются журналы **auth.log** и **syslog**, при этом на конкретной системе может храниться **множество дополнительных журналов** в зависимости от **запущенных на данном хосте служб**.



Описание механизмов закрепления Linux

При реализации злоумышленником доступа к системе одной из его промежуточных целей является **закрепление** в атакованной системе для того, чтобы **сохранить полученный доступ** при перезагрузке системы, действиях **Blue Team** при реагировании или иных обстоятельствах, которые могут повлиять на доступ к атакованной системе. Это характерно в том числе для устройств под управлением ОС семейства Linux.

Наиболее полный перечень возможных техник для закрепления в Linux может быть представлен, если открыть **MITRE ATT&CK Matrix для Linux**:



MITRE | ATT&CK®

MATRICES

Enterprise

PRE

Windows

macOS

Linux

Cloud

Network

Containers

Mobile

ICS

Initial Access

8 techniques

Execution

8 techniques

Persistence

16 techniques

Drive-by Compromise

Exploit Public-Facing Application

External Remote Services

Hardware Additions

Phishing (3)

Supply Chain Compromise (3)

Trusted Relationship

Valid Accounts (3)

Command and Scripting Interpreter (4)

Exploitation for Client Execution

Inter-Process Communication

Native API

Scheduled Task/Job (3)

Software Deployment Tools

System Services

User Execution (2)

Account Manipulation (1)

Boot or Logon Autostart Execution (2)

Boot or Logon Initialization Scripts (1)

Browser Extensions

Compromise Client Software Binary

Create Account (2)

Create or Modify System Process (1)

Event Triggered Execution (3)

External Remote Services

Hijack Execution Flow (1)

Фрагмент матрицы для Linux

Кроме того, более конкретный перечень механизмов закрепления может быть представлен в схеме:

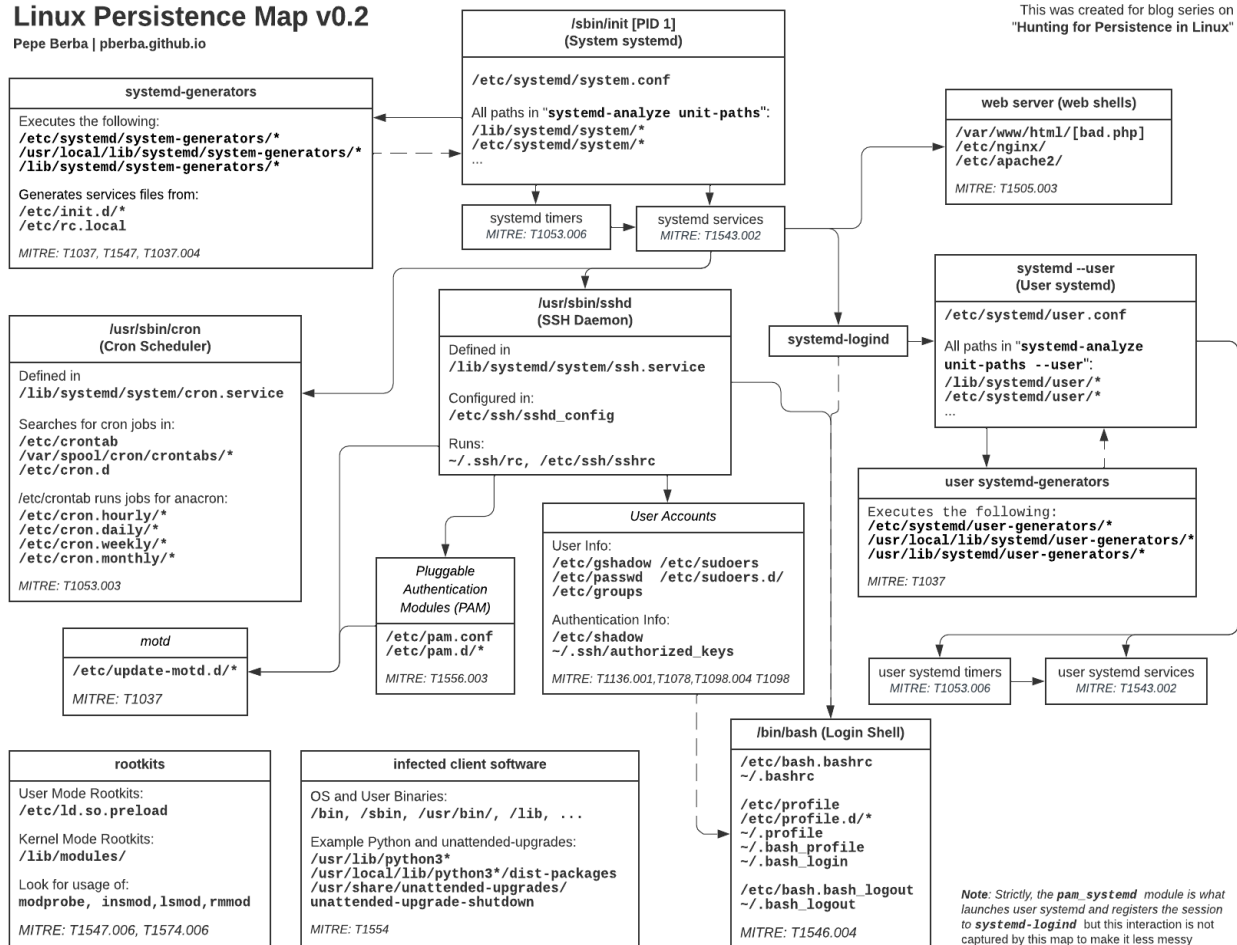
<https://pberba.github.io/assets/posts/common/20220201-linux-persistence.png>



Linux Persistence Map v0.2

Pepe Berba | pberba.github.io

This was created for blog series on
"Hunting for Persistence in Linux"



Механизмы закрепления в Linux

При этом наиболее популярными техниками из описанных для реализации механизма закрепления являются:

rc скрипты;

init скрипты;



```
cron;  
login shell.
```

Несмотря на то, что для инициализации системных компонентов при запуске Linux в настоящее время используется `systemd`, в файл `/etc/rc.local` может быть записан скрипт, который будет **выполнен при запуске**.

`Init.d` это старый механизм Linux, предназначенный для запуска различных служб и задач при запуске системы. В некоторых версиях Linux данный механизм уже не используется и оставлен для **обратной совместимости**.

Скрипты для запуска расположены по пути `/etc/init.d`.

Пример вредоносного скрипта, который может храниться в `init.d`:




```

cat > /etc/init.d/bad-init-d << EOF
#!/bin/sh
### BEGIN INIT INFO
# Provides:          bad-init-d
# Required-Start:    $local_fs $network $syslog
# Required-Stop:     $local_fs $network $syslog
# Default-Start:     2 3 4 5
# Default-Stop:      0 1 6
### END INIT INFO

do_start()
{
    start-stop-daemon --start \
        --pidfile /var/run/init-daemon.pid \
        --exec /opt/backdoor.sh \
        || return 2
}

case "$1" in
    start)
        do_start
        ;;
    esac
EOF

```

Пример вредоносного скрипта в init.d

Cron является наиболее популярным средством создания отложенных задач в Linux. Для поиска признаков закрепления посредством данного механизма могут использоваться следующие пути:

/etc/crontab

/etc/cron.d/*

/etc/cron.{hourly,daily,weekly,monthly}/*

/var/spool/cron/crontab/*



Командная оболочка имеет несколько конфигурационных скриптов, которые запускаются при старте или закрытии Linux Shell:

/etc/profile - общесистемный файл, запускающийся при старте командной оболочки при входе в систему;

/etc/profile.d/ - все расположенные .sh файлы запускаются при старте командной оболочки при входе в систему;

/etc/bash.bashrc - также общесистемный файл, запускающийся при старте командной оболочки при входе в систему;

/etc/bash.bash_logout - запускается при выходе из командной оболочки;

~/.bashrc - стартует при запуске интерактивного окружения пользователя;

~/.bash_profile, **~/.bash_login**, **~/.profile** - также специфичные для конкретного пользователя скрипты

~/.bash_logout - запускается при закрытии сессии пользователя.

Резюме

В ОС семейства Linux присутствует множество способов закрепления, при этом необходимо в первую очередь обращать внимание на самые распространенные и характерные для большинства ОС семейства Linux.

Резюме урока

Анализ ОС семейства Linux требует специфичных знаний о механизмах, которые могут быть вовлечены во вредоносную активность. В первую



очередь это касается файловой активности, журналов событий и различных средств автозапуска.

Навыки анализа Linux при проведении РКИ являются необходимыми, так как часто устройства под управлением Linux являются входными точками в сетевую инфраструктуру из-за разворачиваемых именно на этом семействе ОС специфичных сервисов.

